Agentic RAG

Production-Ready Retrieval-Augmented Generation
Built with LangGraph

Pattern	**Corrective RAG (CRAG)**
Orchestration	**LangGraph**
LLM	**Groq — llama-3.3-70b-versatile**
Embeddings	**HuggingFace — all-MiniLM-L6-v2**
Vector Store	**FAISS (Local)**
Language	**Python 3.11+**

This document presents an Agentic RAG system that goes beyond simple retrieval. It evaluates, corrects, and verifies every step of the pipeline before returning an answer — making it suitable for production deployments.

- ✓ Self-correcting retrieval with LLM-as-judge relevance grading
- ✓ Hallucination detection before every response
- ✓ Query rewriting for improved semantic search
- ✓ Infinite loop prevention with retry counters
- ✓ Fully free stack — no OpenAI billing required

What is Corrective RAG?

Traditional RAG follows a fixed linear pipeline: retrieve documents, then generate an answer. It blindly trusts whatever is retrieved, even if the documents are irrelevant or the answer is hallucinated. This makes traditional RAG unreliable for production use.

Traditional RAG	Corrective RAG (This Project)
Retrieve → Generate	Retrieve → Grade → Correct → Generate → Verify
No quality checks	LLM-as-judge at every step
Hallucinates silently	Detects and rejects hallucinations
No self-correction	Rewrites query when retrieval fails
Single pass	Self-correcting loop with retry limit

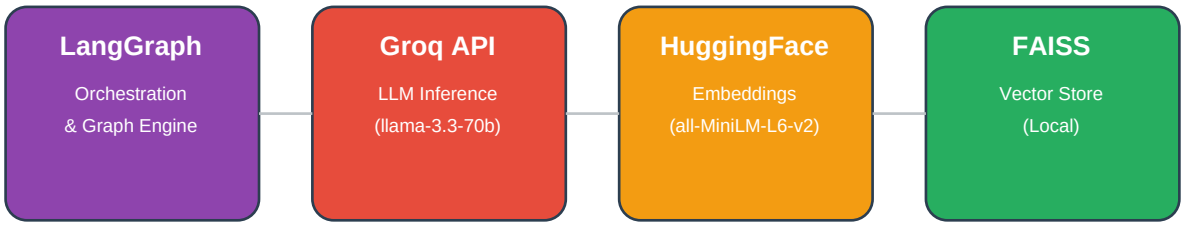
Why Agentic?

This system is called Agentic because the LLM acts as an evaluator and decision-maker — not just a generator. At every stage, an LLM agent judges the quality of retrieved documents, the faithfulness of the generated answer, and whether the answer actually resolves the question. Based on these judgements, the graph dynamically routes to correction or completion.

RAG Evolution

Level	Pattern	Key Feature
1	Basic RAG	Retrieve → Generate
2	Advanced RAG	Better chunking, hybrid search
3	Corrective RAG	Self-evaluation + correction ← You are here
4	Self-RAG	LLM decides whether to retrieve
5	Agentic RAG	Multiple agents, tools, planning

Tech Stack



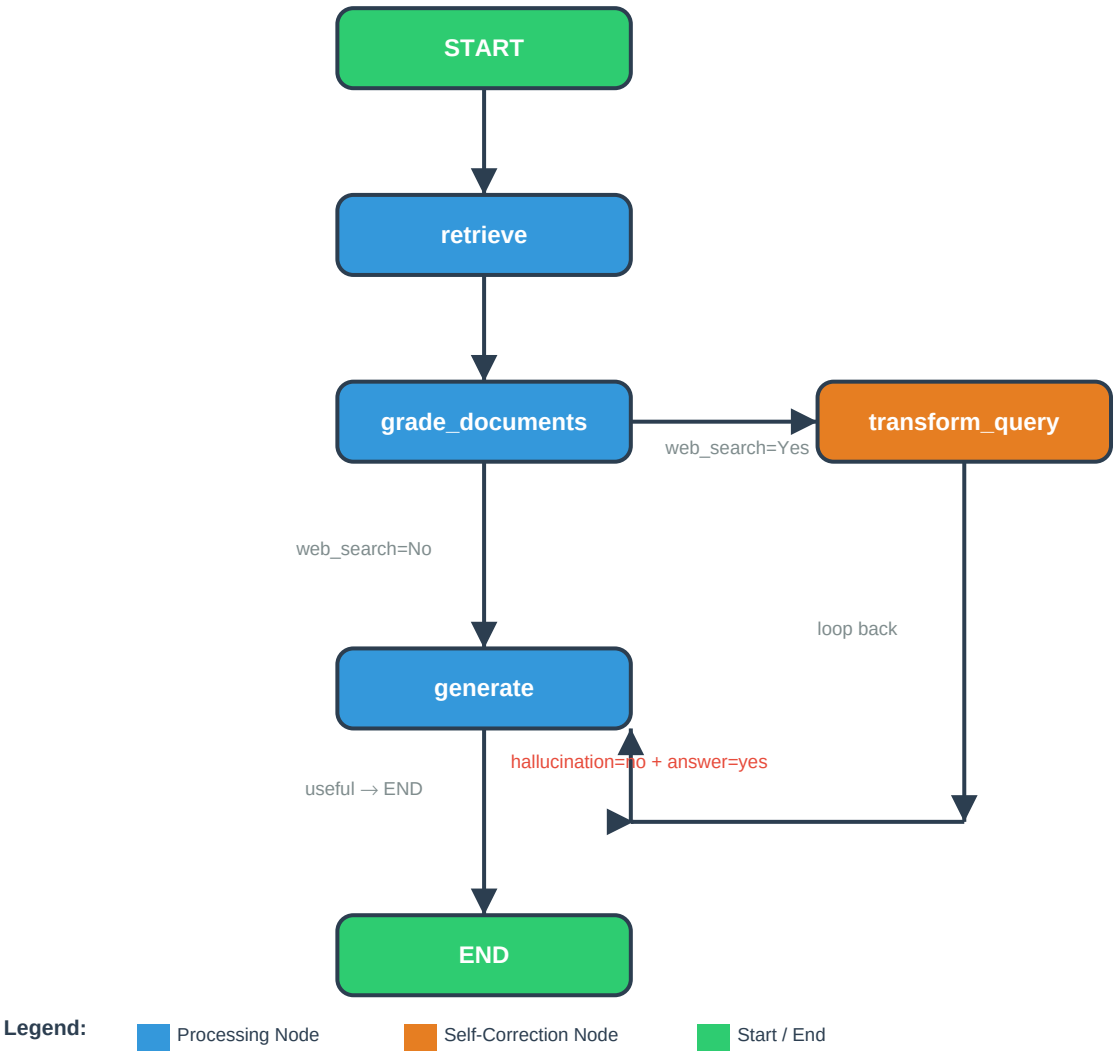
Component	Technology	Role	Cost
Orchestration	LangGraph 0.2+	Graph engine, state management, conditional routing	**Free**
LLM	Groq API llama-3.3-70b	Generation, relevance grading, hallucination & answer grading	**Free tier**
Embeddings	HuggingFace all-MiniLM-L6-v2	Convert text to 384-dim vectors for semantic similarity search	**Free / Local**
Vector Store	FAISS	Store and search 42 document vectors with cosine similarity	**Free / Local**
Document Loader	LangChain Docx2txtLoader	Load Word documents and extract text content	**Free**
Text Splitter	RecursiveCharacter TextSplitter	Split documents into 500-char chunks with 100-char overlap	**Free**
Env Management	python-dotenv	Securely load API keys from .env file	**Free**

Why This Stack?

This stack was chosen to be entirely free and locally runnable — no OpenAI billing, no cloud vector database costs. Groq provides extremely fast LLM inference on its free tier. HuggingFace sentence transformers run locally after a one-time 90MB download. FAISS stores vectors on disk with zero infrastructure cost. LangGraph provides the conditional routing and state management that makes self-correction possible.

Architecture & Workflow

The system is modelled as a directed graph with nodes as processing steps and edges as conditional transitions. State flows through the graph as a shared TypedDict object that every node can read from and write to.



State Field	Type	Set By	Purpose
question	str	User input	Original user question
documents	List	retrieve node	Retrieved and filtered chunks
generation	str	generate node	Final LLM answer
web_search	str (Yes/No)	grade_documents	Trigger query rewrite flag
retries	int	transform_query	Loop prevention counter (max 3)

Production-Ready Features

Relevance Grading
LLM judges each chunk before generation

Hallucination Check
Verifies answer is grounded in retrieved documents

Query Rewriting
Self-corrects bad queries to improve retrieval

Answer Grading
Validates answer resolves the user question

Retry Limiting
Prevents infinite loops with max retry counter

State Management
Shared state across all nodes via TypedDict

Node Descriptions

Node	Type	LLM Call?	Description
retrieve	Processing	No	Vector similarity search, returns top 4 chunks from FAISS
grade_documents	Evaluation	Yes (per chunk)	Filters irrelevant chunks, sets web_search flag if all fail
generate	Generation	Yes	Produces grounded answer from filtered context using RAG promp
transform_query	Correction	Yes	Rewrites question to improve semantic match, increments retries
decide_to_generate	Router	No	Routes to generate or transform_query based on web_search flag
grade_generation	Router	Yes (x2)	Checks hallucination then answer quality, routes to END or retry

LLM-as-Judge Pattern

A key production technique used in this system is LLM-as-judge — using a separate LLM call to evaluate the quality of another LLM call. Three separate judges are used: a relevance judge (is this chunk useful?), a hallucination judge (is this answer grounded?), and an answer judge (does this answer resolve the question?). Each judge uses Pydantic structured output to return a strict binary score — eliminating ambiguous free-text responses.

Project Structure & Results

Project File Structure

File	Purpose
ingestion.py	Load Word doc, chunk text, create and save FAISS vector store
graph_state.py	TypedDict schema defining the 5 shared state fields
nodes.py	All node functions: retrieve, grade, generate, rewrite
graph.py	LangGraph workflow: nodes, edges, router functions, compile
main.py	Entry point: user input loop, stream graph, display answer
data/rag_guide.docx	Source document — the RAG guide itself
vectorstore/faiss_index/	Persisted FAISS index (index.faiss + index.pkl)
.env	GROQ_API_KEY stored securely outside source code

Sample Results

Question	Graph Path	Outcome
What is document chunking?	retrieve → grade → generate → END	Direct answer in 1 pass
What is query rewriting in RAG?	retrieve → grade → generate → END	Direct answer in 1 pass
Who is Naveen?	retrieve → grade → rewrite (x3) → END	Clean I don't know response
What is hallucination grading?	retrieve → grade → generate → verify → END	Grounded answer returned

Key Learnings

→ LangGraph conditional edges are the core of self-correcting pipelines

→ LLM-as-judge with Pydantic structured output is more reliable than free-text evaluation

→ Retry counters must sit in shared state to prevent infinite loops across all paths

→ web_search flag should only trigger when zero relevant documents are found, not on partial failures

→ Embedding model loads once per process — run as a long-lived server in production

→ Chunking quality directly determines retrieval quality — chunk size and overlap matter